Python from Scratch: Notes & Modules

By Yash Rawat. Every topic has concise notes plus 10 programs. Type each program yourself, run it, then change it.

Roadmap

Module	Topics
1. Foundations (this doc)	Setup and print, variables, data types, operators, input and type conversion
2. Control Flow	if/elif/else, loops, break/continue, patterns
3. Data Structures	list, tuple, set, dictionary
4. Strings in Depth	methods, slicing, formatting
5. Functions	arguments, return, scope, lambda, recursion
6. Files and Exceptions	file I/O, try/except, JSON, CSV
7. OOP	classes, inheritance, dunder methods
8. Modules and Standard Library	import, pip, venv, datetime, random, os
9. Advanced Python	comprehensions, generators, decorators, context managers
10. Backend and Projects	Flask/FastAPI, REST APIs, database, mini projects
11. Data Basics	NumPy, Pandas

How to use: read the notes, run all 10 programs, then do the practice tasks at the end of the module. Tell me when you finish and I will give you Module 2.

Module 1: Foundations

Setup

- Install Python 3.12 or newer from python.org (tick "Add to PATH" on Windows).
- Check: `python --version` (or `python3 --version`).

- **REPL:** type `python` to experiment line by line. Exit with `exit()`.
- **Script:** save code as `hello.py` and run `python hello.py`.
- Editor: VS Code with the Python extension.
- Python is **interpreted**, **dynamically typed**, and uses **indentation** (4 spaces) instead of braces.

Topic 1.1: Output with print() and Comments

Notes

- `print(*objects, sep=' ', end='\n')` writes to the screen.
- `sep` controls the separator between items; `end` controls what is printed at the end (default newline).
- Strings can use `'single'`, `"double"` or `'''triple'''` quotes (triple allows multiple lines).
- Escape characters: `\n` newline, `\t` tab, `\\` backslash, `\'` and `\"` quotes.
- Raw string `r"..."` ignores escapes (useful for Windows paths and regex).
- Comments: `# single line`; triple-quoted strings are often used as docstrings.

Programs

```
# 1. Hello World
print("Hello, World!")
```

```
# 2. Printing multiple values
print("Python", "is", "fun")
print("Age:", 21, "Marks:", 8.35)
```

```
# 3. sep parameter
print("2026", "11", "06", sep="-")
print("yash", "gmail.com", sep="@")
```

```
# 4. end parameter
print("Loading", end="...")
print("done")
print("A", end=" ")
print("B", end=" ")
print("C")
```

```
# 5. Escape sequences
print("Name:\tYash\nCity:\tHaldwani")
print("Path: C:\\Users\\Yash")
```

```
# 6. Quotes inside strings
print("It's a nice day")
print('She said "Hi"')
print("He said \"Bye\"")
```

```
# 7. Triple-quoted multi-line string
print('''Line one
Line two
Line three''')
```

```
# 8. Repeating strings to draw
print("*" * 20)
print("=" * 20)
print("-" * 20)
```

```
# 9. A simple banner using f-strings and comments
name = "Yash"          # variable
print("+" + "-" * 18 + "+")
print(f"|  Hello, {name:<8}|")
print("+" + "-" * 18 + "+")
```

```
# 10. Raw strings and docstring
"""This is a docstring-style comment.
It can span many lines."""
print(r"C:\new\table")   # backslashes are kept as typed
print("C:\new\table")    # \n and \t are interpreted
```

Topic 1.2: Variables

Notes

- A variable is a **name bound to an object** in memory. Assignment uses `=`.
- No declaration needed; type comes from the value (dynamic typing) and can change.
- **Naming rules:** letters, digits, underscore; cannot start with a digit; no spaces; case-sensitive; cannot be a keyword (`if`, `for`, `class`, …).
- **Style (PEP 8):** `snake_case` for variables, `UPPER_CASE` for constants (by convention only).
- `id(x)` gives the object's identity; `type(x)` gives its type; `del x` removes the name.

Programs

```
# 1. Basic assignment
name = "Yash"
age = 21
cgpa = 8.35
print(name, age, cgpa)
```

```
# 2. Multiple assignment
a, b, c = 10, 20, 30
print(a, b, c)
```

```
# 3. Same value to many variables
x = y = z = 0
print(x, y, z)
```

```
# 4. Swapping without a temp variable
a, b = 5, 9
a, b = b, a
print("a =", a, "b =", b)
```

```
# 5. Dynamic typing: type changes with value
v = 10
print(v, type(v))
v = "ten"
print(v, type(v))
v = 10.0
print(v, type(v))
```

```
# 6. Variables are references (id)
a = 100
b = a
print(id(a) == id(b))   # True: same object
b = 200
print(id(a) == id(b))   # False: b now points elsewhere
print(a, b)
```

```
# 7. Valid vs invalid names
print("user_name".isidentifier())   # True
print("2cool".isidentifier())       # False
print("my var".isidentifier())      # False
print("_hidden".isidentifier())     # True
```

```
# 8. Keywords you cannot use as names
import keyword
print(len(keyword.kwlist))
print(keyword.kwlist)
print(keyword.iskeyword("class"))   # True
```

```
# 9. Constants by convention and case sensitivity
PI = 3.14159
MAX_USERS = 100
name = "A"
Name = "B"
print(PI, MAX_USERS, name, Name)    # name and Name are different
```

```
# 10. del and NameError
city = "Haldwani"
print(city)
del city
try:
    print(city)
except NameError as e:
    print("Error:", e)
```

Topic 1.3: Data Types

Notes

Type	Example	Mutable?
`int`	`42`, `1_000_000`	No
`float`	`3.14`, `2e3`	No
`complex`	`3+4j`	No
`bool`	`True`, `False`	No
`str`	`"hello"`	No
`NoneType`	`None`	No
`list`	`[1, 2]`	**Yes**
`tuple`	`(1, 2)`	No
`set`	`{1, 2}`	**Yes**
`dict`	`{"a": 1}`	**Yes**

- `int` has unlimited size. `float` is a 64-bit binary approximation (so `0.1 + 0.2 != 0.3`).
- **Falsy values:** `0`, `0.0`, `""`, `[]`, `()`, `{}`, `set()`, `None`, `False`. Everything else is truthy.
- `type(x)` shows the type; `isinstance(x, int)` checks it (preferred).

Programs

```
# 1. int: unlimited size and readable literals
big = 2 ** 100
print(big)
print(1_000_000 + 1)
print(type(big))
```

```
# 2. float and precision
print(0.1 + 0.2)                  # 0.30000000000000004
print(round(0.1 + 0.2, 2))        # 0.3
print(2e3, type(2e3))             # 2000.0 <class 'float'>
```

```
# 3. complex numbers
z = 3 + 4j
print(z.real, z.imag)
print(abs(z))                     # magnitude 5.0
print(z * (1 - 2j))
```

```
# 4. bool is a subclass of int
print(True + True)                # 2
print(int(True), int(False))
print(isinstance(True, int))      # True
```

```
# 5. str basics: indexing and immutability
s = "Python"
print(s[0], s[-1], len(s))
try:
    s[0] = "J"
except TypeError as e:
    print("Error:", e)
```

```
# 6. None: the absence of a value
result = None
print(result, type(result))
print(result is None)             # use 'is' with None
```

```
# 7. type() vs isinstance()
x = 3.5
print(type(x))
print(isinstance(x, float))
print(isinstance(x, (int, float)))   # check against several types
```

```
# 8. Number systems
n = 25
print(bin(n), oct(n), hex(n))
print(0b11001, 0o31, 0x19)           # all equal 25
print(int("11001", 2), int("ff", 16))
```

```
# 9. Truthy and falsy values
values = [0, 1, "", "hi", [], [0], None, 0.0, {}, " "]
for v in values:
    print(repr(v), "->", bool(v))
```

```
# 10. Mutable vs immutable (preview)
nums = [1, 2, 3]
nums[0] = 99
print(nums)                          # list changed in place
t = (1, 2, 3)
try:
    t[0] = 99
except TypeError as e:
    print("Tuple error:", e)
print(type(nums), type(t), type({1, 2}), type({"a": 1}))
```

Topic 1.4: Operators

Notes

- **Arithmetic:** `+ - * / // % **` (`/` always gives float; `//` floors; `%` remainder).
- **Comparison:** `== != > < >= <=` (can be chained: `1 < x < 10`).
- **Logical:** `and`, `or`, `not` (short-circuit; they return one of the operands).
- **Assignment:** `= += -= *= /= //= %= **=`.
- **Bitwise:** `& | ^ ~ << >>`.
- **Identity:** `is`, `is not` (same object). **Membership:** `in`, `not in`.

- **Precedence (high to low):** `**`, unary `-`, `* / // %`, `+ -`, comparisons, `not`, `and`, `or`. Use parentheses when unsure.

Programs

```
# 1. Arithmetic operators
a, b = 17, 5
print(a + b, a - b, a * b)
print(a / b, a // b, a % b, a ** b)
```

```
# 2. Floor division and modulus with negatives
print(-17 // 5)     # -4 (floors toward negative infinity)
print(-17 % 5)      # 3  (result has the sign of the divisor)
print(17 // -5, 17 % -5)
print(divmod(17, 5))
```

```
# 3. Power and roots
print(2 ** 10)
print(9 ** 0.5)         # square root
print(27 ** (1 / 3))    # cube root (approx)
print(pow(2, 5, 7))     # (2**5) % 7
```

```
# 4. Comparison and chaining
x = 7
print(x == 7, x != 7, x > 5, x <= 6)
print(1 < x < 10)        # chained: True
print("apple" < "banana")  # alphabetical: True
```

```
# 5. Logical operators and short-circuit
print(True and False, True or False, not True)
print(0 or "default")        # 'default'
print("hi" and "there")      # 'there'
print(5 and 0)               # 0
```

```
# 6. Augmented assignment
score = 10
score += 5
score -= 3
score *= 2
score //= 5
score **= 2
print(score)
```

```
# 7. Bitwise operators
a, b = 12, 10            # 1100 and 1010
print(a & b, a | b, a ^ b)
print(~a, a << 1, a >> 2)
print(bin(a & b))
```

```
# 8. is vs ==
a = [1, 2, 3]
b = [1, 2, 3]
c = a
print(a == b)    # True: equal values
print(a is b)    # False: different objects
print(a is c)    # True: same object
```

```
# 9. Membership operators
print("y" in "Yash".lower())
print(3 in [1, 2, 3], 5 not in [1, 2, 3])
print("name" in {"name": "Yash"})    # checks keys
```

```
# 10. Precedence and a real use: even/odd and leap year
print(2 + 3 * 4 ** 2)        # 50
print((2 + 3) * 4 ** 2)      # 80
n = 14
print("even" if n % 2 == 0 else "odd")
year = 2028
print((year % 4 == 0 and year % 100 != 0) or year % 400 == 0)
```

Topic 1.5: Input and Type Conversion

Notes

- `input(prompt)` **always returns a string**. Convert it with `int()`, `float()`, etc.
- Conversions: `int()`, `float()`, `str()`, `bool()`, `list()`, `tuple()`, `set()`.
- `int("abc")` raises `ValueError`; handle with `try/except`.
- `int(3.9)` truncates to `3` (does not round).
- **f-strings:** `f"{value:.2f}"` (2 decimals), `{n:>5}` (right align), `{n:05d}` (zero pad), `{n:,}` (thousands separator).

Programs

```
# 1. input returns a string
name = input("Enter your name: ")
print("Hello,", name)
print(type(name))
```

```
# 2. Add two integers
a = int(input("First number: "))
b = int(input("Second number: "))
print("Sum =", a + b)
```

```
# 3. Average of three marks (float input)
m1 = float(input("Mark 1: "))
m2 = float(input("Mark 2: "))
m3 = float(input("Mark 3: "))
print(f"Average = {(m1 + m2 + m3) / 3:.2f}")
```

```
# 4. Type casting examples
print(int("42") + 1)       # 43
print(int(3.99))           # 3 (truncates)
print(float("2.5") * 2)    # 5.0
print(str(100) + " marks")
print(bool(""), bool("0"))   # False True
print(list("abc"), tuple([1, 2]), set([1, 1, 2]))
```

```
# 5. Safe conversion with try/except
text = input("Enter a number: ")
try:
    num = float(text)
    print("You entered", num)
except ValueError:
    print("That is not a valid number.")
```

```
# 6. Multiple values on one line
x, y = map(int, input("Enter two numbers: ").split())
print("Product =", x * y)
nums = list(map(int, input("Enter many numbers: ").split()))
print(nums, "Total =", sum(nums))
```

```
# 7. Formatting output with f-strings
price = 1234.5678
n = 7
print(f"{price:.2f}")
print(f"{price:,.1f}")
print(f"[{n:>5}] [{n:<5}] [{n:^5}] [{n:05d}]")
print(f"{0.256:.1%}")
```

```
# 8. Simple interest calculator
p = float(input("Principal: "))
r = float(input("Rate (% per year): "))
t = float(input("Time (years): "))
si = p * r * t / 100
print(f"Simple interest = {si:.2f}")
print(f"Total amount   = {p + si:.2f}")
```

```
# 9. Temperature converter
c = float(input("Temperature in Celsius: "))
f = c * 9 / 5 + 32
k = c + 273.15
print(f"{c}°C = {f:.1f}°F = {k:.2f} K")
```

```
# 10. Mini bill generator
item = input("Item name: ")
qty = int(input("Quantity: "))
rate = float(input("Rate per unit: "))
subtotal = qty * rate
gst = subtotal * 0.18
print("-" * 30)
print(f"{item:<12}{qty:>4} x {rate:>8.2f}")
print(f"{'Subtotal':<18}{subtotal:>10.2f}")
print(f"{'GST (18%)':<18}{gst:>10.2f}")
print(f"{'Total':<18}{subtotal + gst:>10.2f}")
print("-" * 30)
```

Module 1 Practice Tasks

Write these yourself without looking at the programs above:

1. Print your name, college and city on separate lines, aligned with tabs.
2. Swap two numbers entered by the user.
3. Take a 3-digit number and print its last digit and the remaining digits (hint: `% 10`, `// 10`).
4. Convert seconds entered by the user into hours, minutes and seconds (use `//` and `%`).
5. Calculate the area and circumference of a circle for a given radius.
6. Take the length and width of a rectangle and print area and perimeter with 2 decimals.
7. Check whether a number is divisible by both 3 and 5 using logical operators.
8. Take three numbers and print the largest using only comparison and logical operators.
9. Calculate the final price after a discount percentage and then add 18% GST.
10. Read a full name and age, and print: `Yash Rawat will be 25 in 4 years`.

Module 1 Cheat Sheet

```
print(a, b, sep=", ", end="\n")     # output
x = int(input("> "))                # input + convert
type(x), isinstance(x, int)         # type checks
+ - * / // % **                     # arithmetic
== != < > <= >=                     # comparison
and  or  not                        # logical
is  is not  in  not in              # identity, membership
f"{val:.2f}"                        # formatting
```

Common mistakes: forgetting that `input()` returns a string, using `=` instead of `==`, mixing tabs and spaces for indentation, and comparing floats with `==`.